



重慶理工大學

实验报告

课程 _____ 嵌入式系统及应用 _____

题目 _____ GPIO 输出蜂鸣器发声实验 _____

院系名称 _____

专业 _____ 班级 _____

学生姓名 _____ 学号 _____

任课教师 _____

时 间 _____

一、实验设备与软件环境

- 设备：
- ① PC 电脑，至少需要 2 个 Type-A 的 USB 插口；
 - ② ALIENTEK MiniSTM32 实验套件；
 - ③ miniIO-V2 接口板；
 - ④ 杜邦线
- 环境：
- macOS 26.3；CLion 2025.3；CMake、OpenOCD、arm-none-eabi-gdb 工具链

二、实验任务

- 通过按键及按键组合选择输出以下声音：
- ① 模拟枪声、救护车警报声、消防车警报声演示程序，阅读源程序、下载验证；
 - ② 播放音乐程序：设计音乐音调（频率）和节奏（节拍）存储格式，编写音乐播放子程序，至少用 2 首歌曲验证设计程序。

三、实验原理图及分析

3.1 实验连线示意图

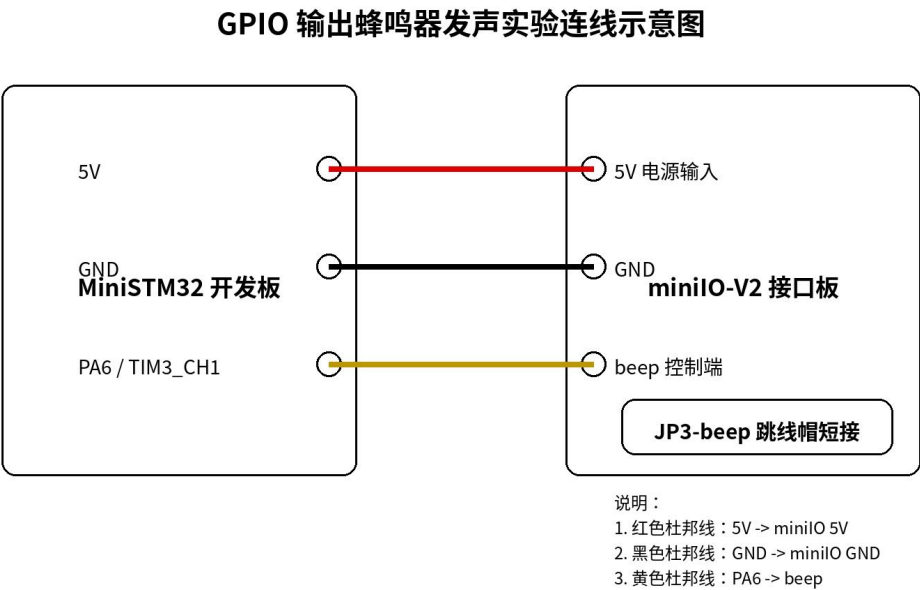


图 3-1 MiniSTM32 与 miniIO-V2 的蜂鸣器实验连线示意图

3.2 实验连线文字分析

连接对象	MiniSTM32 端	miniIO 端	作用说明
电源正极	5V	5V 电源输入	为蜂鸣器驱动级供电
公共地	GND	GND	建立统一参考地，保证信号可靠

控制信号	PA6 / TIM3_CH1	beep	输出与音高对应的 PWM/方波信号
跳线帽	—	JP3-beep 短接	接通蜂鸣器供电，否则蜂鸣器不工作

本实验实际连线只需要 3 根杜邦线：5V、GND 和 PA6 控制线。指导书说明 miniIO 板上的蜂鸣器为无源电磁式蜂鸣器，beep 引脚并不直接驱动蜂鸣器本体，而是经三极管驱动级放大电流后带动蜂鸣器发声，因此除了控制线外，还必须保证电源线和跳线帽状态正确。

PA6 在实际程序中没有按普通推挽 GPIO 翻转，而是被配置为 TIM3_CH1 的复用推挽输出。这样可以把频率稳定地交给定时器 PWM 产生，既减小 CPU 负担，也使音调比单纯软件翻转更稳定。

四、发声原理及程序控制方法

4.1 蜂鸣器发声原理

miniIO 扩展板使用的是无源电磁式蜂鸣器。无源蜂鸣器内部没有自激振荡源，只有在线圈两端持续施加交变电压时才会发声；如果一直给直流电平，只会保持吸合而不会形成可听见的连续音调。蜂鸣器的音高由输入信号频率决定，频率越高，音调越高；频率越低，音调越低。一个音符持续多长时间，则对应音乐中的节拍长度。

4.2 程序控制方法

程序将 PA6 配置为 TIM3_CH1 复用推挽输出，通过定时器直接产生 PWM。程序先把 TIM3 的计数频率设为 10MHz，然后在 `set_beep_hz()` 中根据 `period_ticks = 10000000 / frequency_hz` 计算 ARR，再根据占空比计算 CCR1，从而输出指定频率的波形。

当 `frequency_hz` 为 0 时，程序把 TIM3 的比较值置 0，蜂鸣器停止发声；当 `frequency_hz` 不为 0 时，TIM3 按新的周期自动输出波形。代码中用 `VOLUME_PER_TEN_THOUSAND` 对占空比做了进一步约束，以降低无源蜂鸣器的实际音量，避免声音过于尖锐。

4.3 音乐音调与节拍的存储格式设计

字段	含义	示例
freq	音调频率，单位 Hz；0 表示休止符	330 / 440 / 0
dur	持续时间，单位 us	326922 / 216345
注释行	以 # 开头，解析时整行跳过	# Source: xxx.mid
结束条件	到达数据末尾或遇到字符串结束符	\0

实际乐谱数据保存在 `music_data.c` 中，本质上是一份文本化数组：每一行先写频率，再

写持续时间。play_music_file() 逐行解析数据，自动跳过空白和注释，再调用 play_note() 播放当前音符。这种结构虽然简单，但可读性好、修改方便，后续若需要扩展到多首歌曲，只需增加新的乐谱数据文件或切换数据指针即可。

五、程序流程图

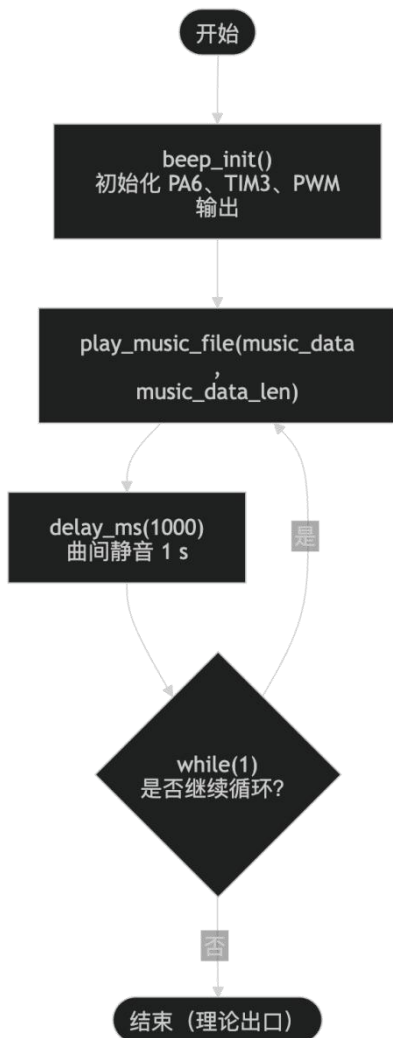


图 5-1 主程序流程图

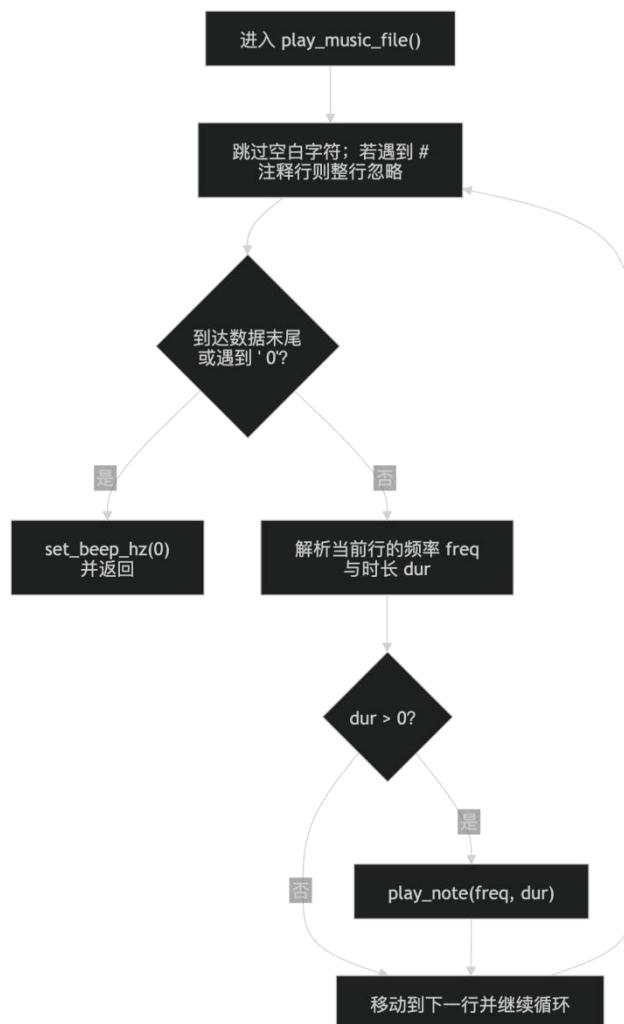


图 5-2 乐谱解析与音符播放流程图

主程序结构先完成 beep_init()，随后进入 while(1) 死循环，不断播放乐谱数据，曲目结束后再静音约 1s 并重新开始。

在播放函数内部，程序先跳过空白字符和以 # 开头的注释行，再解析当前行的频率与持续时间。如果 dur > 0，则调用 play_note() 播放；如果 freq 为 0，则 play_note() 会进入休止符分支，只保持静音延时。全部数据处理完成后，再统一关闭蜂鸣器输出。

六、主要程序分析

6.1 主要文件与函数分工

文件/函数	所在文件	主要作用
delay_ticks / delay_us / delay_ms	main.c	基于 SysTick 完成微秒与毫秒级延时
beep_init	main.c	初始化 PA6、TIM3 和 PWM 输出通道
set_beep_hz	main.c	根据目标频率计算 ARR/CCR, 设置蜂鸣器音高
play_note	main.c	播放单个音符或休止符
play_music_file	main.c	解析文本乐谱并依次调用 play_note
music_data[]	music_data.c	保存具体乐谱数据与注释信息

6.2 底层初始化函数

beep_init() 先开启 GPIOA 和 TIM3 的时钟, 然后把 PA6 配置为 GPIO_Mode_AF_PP, 即复用推挽输出; 接着把 TIM3 的分频值设置为 (SystemCoreClock / 10000000) - 1, 使计数器工作在 10MHz; 最后把通道 1 配置为 PWM1 模式并启动 TIM3。此后, 只需要更新定时器参数, 就可以连续输出不同频率的驱动波形。

6.3 频率控制函数

set_beep_hz() 是本实验最核心的控制函数。它把“目标频率”转换为“定时器周期”: $\text{period_ticks} = 10000000 / \text{frequency_hz}$; 随后调用 TIM_SetAutoreload() 设置周期, 再通过 TIM_SetCompare1() 设置比较值。frequency_hz 为 0 时直接关闭输出, 这样同一个函数就同时兼顾了“发声”和“休止”两种情况。

6.4 解析函数

play_music_file() 顺序扫描字符数组, 一边过滤空格、换行、Tab 和注释行, 一边读取频率与持续时间两个十进制数字。解析出的 dur 只要大于 0, 就进入 play_note()。

七、实验步骤

- ① 分析软硬件原理、方法;
- ② 编写源程序;
- ③ 下载调试程序;
- ④ 修改源程序;
- ⑤ 撰写实验报告。

八、程序调试及结果分析

8.1 调试结果

检查项	现象/结果	分析
蜂鸣器开始按		
上电后运行	music_data[] 内容播放旋律	main() 初始化后立即进入播放函数
遇到 0 频率数据	对应时段保持静音	play_note() 在 freq==0 时进入休止符分支
乐曲播放结束	关闭蜂鸣器，静音约 1s 后重新播放	play_music_file() 结束后 set_beep_hz(0)，main() 中 delay_ms(1000)
修改频率数据	音高发生对应变化	ARR 由 $period_ticks = 10000000 / f$ 重新计算
修改持续时间	节拍长短发生变化	dur 决定该音符或休止符持续时间

8.2 调试过程中重点问题及处理方法

(1) 蜂鸣器音量过大

处理方法：用较小占空比输出，从而在保证可听见的前提下降低音量。

处理结果：将占空比设为 10U 左右听感较为合适。

(2) 某些乐曲速度变快、节奏混乱

问题原因：需要装入 SysTick->LOAD 的计数值大于 24 位寄存器所能表示的最大值 0xFFFFF。

处理方法：delay_ticks() 先把总 ticks 切成不超过最大值的多个片段，再分段完成延时。

处理结果：播放效果好了一些，但仍与原曲效果存在差异。

九、思考题

9.1 软件延时的特点

优点：实现直接、容易理解，不需要额外配置专用硬件。

缺点：CPU 在延时期间基本处于忙等待状态，不能高效并发处理其他任务；精度会受到主频和中断响应等未知影响。

9.2 可替代的软件/硬件实现方式

- (1) 使用通用定时器中断或输出比较，在中断中翻转 IO 或更新比较值
- (2) 直接使用定时器 PWM 输出频率，把节拍交给另一个定时器或 SysTick 处理（本工程实现）